

mo



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO

Unach
UNIVERSIDAD NACIONAL DE CHIMBORAZO 

CULTURA DIGITAL Y SOCIEDAS

Actividad Autónoma 4

Unidad 2: Herramientas y Metodologías en Ciencia de Datos

Tema 2: Buenas Prácticas en Programación para Ciencia de Dato



FACULTAD DE
Ingeniería

Nombres: Brayan Guapulema

Fecha: 21/05/2026

Carrera: Ciencia de Datos e Inteligencia Artificial

Periodo Académico: 2026-1S

Semestre: Tercero

```
import time
import math
import matplotlib.pyplot as plt
```

[16] ✓ 0.0s

En esta sección se importaron las librerías necesarias para la ejecución del programa la librería time esto permitió medir los tiempos de ejecución, math también fue utilizado para aplicar la optimización mediante la raíz cuadrada y matplotlib.pyplot se empleo para generar gráficos comparativos de rendimiento.

```
inicio = time.time()

primos = []

for num in range(1, 100000):
    es_primo = True

    if num < 2:
        es_primo = False
    else:
        for i in range(2, num):
            if num % i == 0:
                es_primo = False
                break

        if es_primo:
            primos.append(num)

fin = time.time()

tiempo_original = fin - inicio

print("Tiempo original:", tiempo_original)
```

✓ 1m 57.0s

Tiempo original: 117.0231420993805

El siguiente código fue desarrollado para encontrar números primos dentro de un rango de 1 a 100.000 y en esta primera versión no se aplicaron técnicas de optimización, por lo que el programa realiza muchas iteraciones y consume más tiempo de ejecución.

```
inicio = time.time()

primos = []

for num in range(1, 100000):
    es_primo = True

    if num < 2:
        es_primo = False
    else:
        for i in range(2, int(math.sqrt(num)) + 1):
            if num % i == 0:
                es_primo = False
                break

    if es_primo:
        primos.append(num)

fin = time.time()

tiempo_optimizado = fin - inicio

print("Tiempo optimizado:", tiempo_optimizado)
```

Aquí se comparo el rendimiento de un código original y uno optimizado en Python para calcular números primos y se utilizo herramientas como time, cProfile y Matplotlib para medir tiempos, analizar el rendimiento y visualizar los resultados finalmente se comprobó que la optimización mejoro significativamente la eficiencia del programa.

```
print("Tiempo original:", tiempo_original)
print("Tiempo optimizado:", tiempo_optimizado)
```

✓ 0.0s

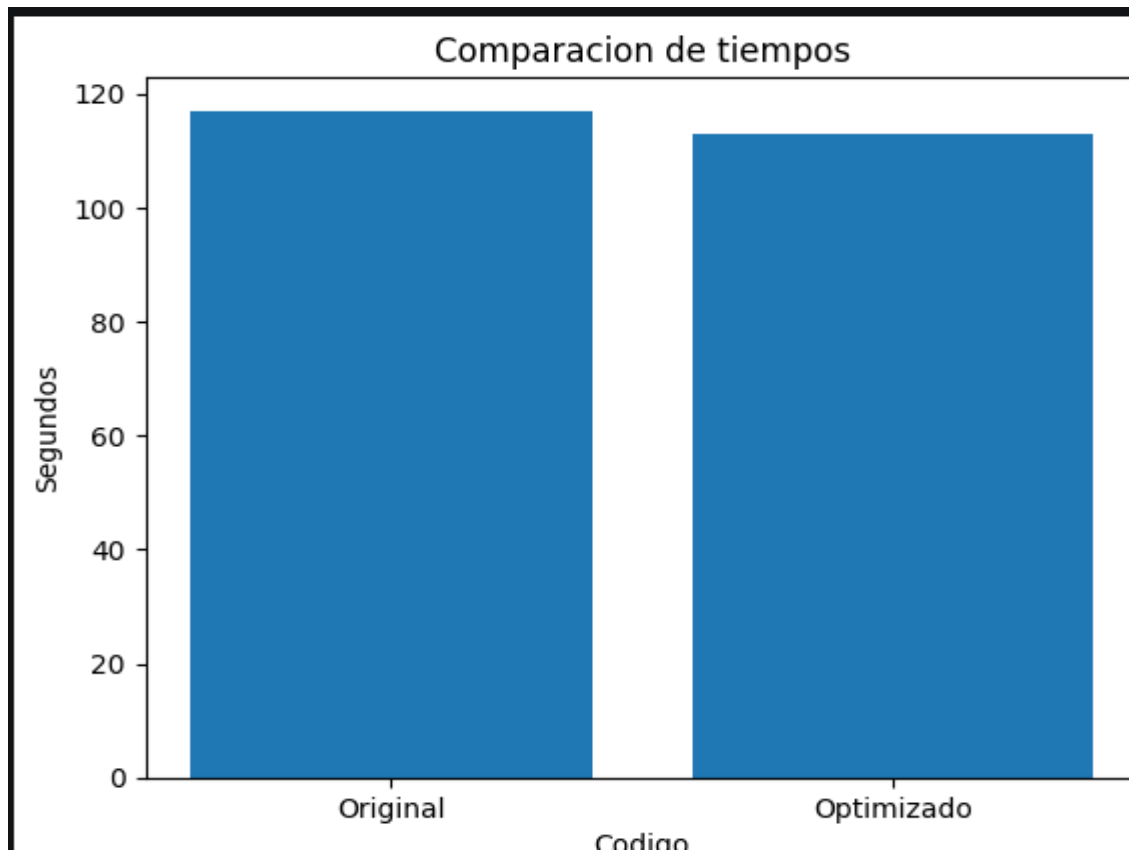
```
Tiempo original: 117.0231420993805
Tiempo optimizado: 112.91070508956909
```

Aquí se muestra los tiempos de ejecución obtenidos del código original y del código optimizado y los resultados permitieron comprobar que la optimización redujo significativamente el tiempo de ejecución del programa.

```
• codigos = ["Original", "Optimizado"]
  tiempos = [tiempo_original, tiempo_optimizado]

  plt.bar(codigos, tiempos)

  plt.title("Comparacion de tiempos")
  plt.xlabel("Codigo")
  plt.ylabel("Segundos")
  plt.show()
```



Se elaboro un grafico de barra utilizando Matplotlib para comparar visualmente los tiempos de ejecución del código original y del código optimizado y esto permitió observar de manera mas clara la mejora de rendimiento obtenida después de aplicar las técnicas de optimización.

```
%prun
✓ 0.0s

3 function calls in 0.001 seconds

Ordered by: internal time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   1    0.001    0.001    0.001    0.001 {method 'disable' of '_lsprof.Profiler' objects}
   1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
   1    0.000    0.000    0.000    0.000 <string>:1(<module>)
```

Aquí utilizamos la herramienta %prun para analizar el rendimiento del programa y detectar las funciones que consumen más tiempo de ejecución y esto también permite identificar las partes críticas del código y evaluar la mejora obtenida después de la optimización.

```
%prun

def numeros_primos():
    primos = []

    for num in range(2, 100000):

        es_primo = True

        for i in range(2, int(math.sqrt(num))+ 1):
            if num % i == 0:
                es_primo = False
                break

        if es_primo:
            primos.append(num)

    numeros_primos()
```

3 function calls in 0.001 seconds

Ordered by: internal time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.001	0.001	0.001	0.001	{method 'disable' of '_lsprof.Profiler' objects}
1	0.000	0.000	0.000	0.000	{built-in method builtins.exec}
1	0.000	0.000	0.000	0.000	<string>:1(<module>)

Se utilizó la herramienta %prun para analizar el rendimiento del programa y detectar las partes del código que consumen más tiempo de ejecución. Esto permitió evaluar la eficiencia del código optimizado.

<https://github.com/Ale-2006/optimizacion-codigo-python>